TYPE HINTS IN PYTHON

Class 04 - Functions | What they mean, why they are useful, and when to use them

Key idea: a type hint describes the expected type, but Python normally does not enforce it at runtime.

1. What is a type hint?

A **type hint** is an annotation that indicates the type of data we expect to receive or return. It helps humans, editors such as VS Code, and static analyzers understand a function's contract.

```
def taylor_exp(x: float, order: int) -> float:
    ...
```

Here **x: float** means we expect a real number, **order: int** means we expect an integer, and **-> float** means the function should return a float.

2. Python does NOT become statically typed

```
def add(a: int, b: int) -> int:
    return a + b

result = add("hello ", "world")
print(result)
```

Even though the parameters are annotated as **int**, Python can still execute this call if the operations inside are valid. Type hints are not a runtime barrier by default.

Type hints = contract/documentation + static analysis. They are not equivalent to C/C++ compile-time type checking.

3. What does Optional[float] mean?

```
from typing import Optional

def taylor_exp(x: float, order: int) -> Optional[float]:
    ...
```

Optional[float] means the function may return a **float** or it may return **None**. This matches our current validation pattern using **return None**.

```
def taylor_exp(x: float, order: int) -> Optional[float]:
    if order < 0:
        return None

    approximation = ...
    return approximation
```

In modern Python it can also be written as **float | None**. On older Python versions, **Optional[float]** is the compatible and explicit form.

4. Important: Optional does NOT mean "optional argument"

Concept	Meaning
Optional[float]	The value may be float or None.
g0=9.80665	The argument may be omitted because a default value exists.

```
def rocket_delta_v(isp: float, g0: float = 9.80665) -> float:
    ...
```

Here **g0** is optional in the call because it has a default value, but its expected type is still **float**. These are different concepts.

5. What are they actually used for?

• Immediate documentation

• Autocomplete and editor assistance in VS Code/Pylance

• Finding errors before running the code

• Safer refactoring

• Easier-to-use APIs and libraries

• Teamwork and large projects

6. Engineering example: a rocket function

```
from typing import Optional

def calculate_rocket_delta_v(
    isp: float,
    initial_mass: float,
    final_mass: float,
    g0: float = 9.80665
) -> Optional[float]:

    if isp < 0 or initial_mass <= 0 or final_mass <= 0:
        return None

    if initial_mass < final_mass:
        return None

    ...
```

By looking only at the function header, we already know it expects three numeric quantities, that **g0** has a default value, and that the result may be a float or None. We do not need to read the whole body to understand the interface.

7. What can VS Code detect?

```
def kinetic_energy(mass: float, speed: float) -> float:
    return 0.5 * mass * speed**2

kinetic_energy("heavy", 120.0)
```

With an analyzer such as Pylance/Pyright, the editor can warn that a **str** is being passed where a **float** was expected. This is a development warning, not a prohibition enforced by the interpreter.

8. Simple comparison with C/C++

Python type hints	C/C++
Annotations for tools and humans.	Types are part of the language type system.
Normally not enforced at runtime.	The compiler checks many incompatibilities.
Can be added gradually.	Types are required in the corresponding declarations.

9. Type hints for variables and containers

```
mass: float = 500000.0
name: str = "Vehicle A"
iterations: int = 100

altitudes: list[float] = [0.0, 10.0, 25.0]
coefficients: dict[str, float] = {"Cd": 0.3, "Cl": 0.8}
```

In scientific code they become especially useful once data stops being simple scalars and lists, dictionaries, arrays, objects, and functions calling each other begin to appear.

10. When are they worth using?

Situation	Recommendation
Short exercise for learning logic	Optional; they may distract.
Reusable module	Very useful.
GitHub / portfolio project	Recommended once the architecture is stable.
API, team, or large codebase	Highly recommended.

For our course: we will omit them in the exercises for now. We will bring them back when we start building modules and larger projects.

11. Learning Tracker

Concept	Status
Type hint	✓
Parameter annotation	✓
Return annotation	✓
Optional / None	✓
Static analysis	✓
Runtime vs static typing	✓
VS Code / Pylance	✓
C/C++ comparison	✓

Key sentence: type hints do not make Python "know" the type at runtime; they make our code communicate its intentions more clearly.